**TRƯỜNG ĐẠI HỌC BÁCH KHOA**
**KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO**

**Báo cáo chuyên sâu**
Môn: Lập trình Đa nền tảng

# Xử lý tác vụ nặng với Isolate (hàm `compute`) để giữ giao diện mượt

Chủ đề 03 – Khối kiến thức 1: Nền tảng Dart nâng cao và bất đồng bộ

**SVTH:** 1. [Họ tên 1 - MSSV]
2. [Họ tên 2 - MSSV]
**GVHD:** [Tên giảng viên]

Đà Nẵng, 2026

# Nội dung báo cáo

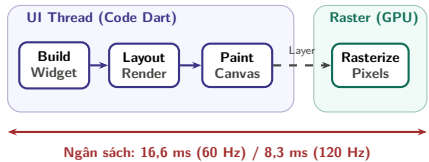
1. **Vấn đề hiệu năng**: UI Jank, Freeze và cơ chế Event Loop trong Dart
2. **Mô hình Isolate**: Cơ chế hoạt động và giải pháp với hàm `compute()`
3. **Hiện thực mã nguồn**: Thiết kế kiến trúc và phân tích mã nguồn
4. **Thực nghiệm & Đánh giá**: Demo ứng dụng, kết quả đo thực tế và kết luận

**Câu hỏi định hướng**

*"Làm sao xử lý tác vụ tính toán nặng mà giao diện Flutter vẫn phản hồi mượt mà?"*

# Frame và ngân sách thời gian

- Mỗi frame có **ngân sách thời gian**: 60 Hz → **16,6 ms**; 120 Hz → **8,3 ms**.
- Một frame đi qua *Build* → *Layout* → *Paint* (**UI thread**, Dart) rồi *Raster* (**Raster thread**, GPU).
- **Jank**: Frame vượt ngân sách → giật khung hình.
- **Freeze**: Nhiều frame liên tiếp bị bỏ → đứng giao diện.
- **Nguyên nhân gốc**: Code Dart nặng chạy **đồng bộ** trên UI thread.

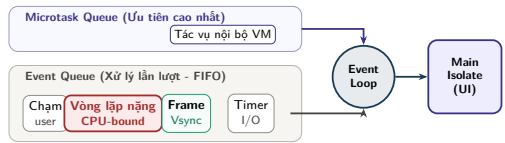



**Khung chốt nguyên nhân**

Nếu tác vụ Dart chiếm UI thread lâu hơn 16,6 ms thì không kịp dựng frame mới → Jank/Freeze.

# Dart đơn luồng: Isolate chính và Event Loop

- Mỗi **Isolate** có 1 luồng Dart và **Event Loop riêng**; Flutter khởi động với **Main Isolate**.
- Hai hàng đợi: **Microtask queue trước**, rồi **Event queue** (chạm, timer, I/O, vẽ frame).
- **Run-to-completion**: Mỗi tác vụ chạy đến khi hoàn tất, không bị ngắt giữa chừng.
- Tác vụ đồng bộ dài → mọi sự kiện sau nó, kể cả vẽ frame, đều phải chờ.




**Bản chất đơ giao diện (UI Freeze)**

Sự kiện vẽ frame nằm trong Event queue nhưng bị chặn bởi một tác vụ đồng bộ chạy quá lâu.

# Minh hoạ Event Loop bằng code

Minh hoạ thứ tự ưu tiên trong Event Loop của Dart (mô hình lý thuyết)

```dart
import 'dart:async';

void main() {
  print('1 sync');
  Future(() => print('4 event')); // event queue
  scheduleMicrotask(() => print('3 microtask')); //
      microtask
  print('2 sync');
}
// Output: 1 sync -> 2 sync -> 3 microtask -> 4
    event
```

- **Code đồng bộ chạy trước (1, 2)**: Nằm trực tiếp trong tác vụ hiện tại của Main Isolate.
- **Microtask queue chạy trước (3)**: Được xử lý xong toàn bộ trước khi Event queue bắt đầu.
- **Event queue chạy sau (4)**: Tác vụ `Future` nằm ở hàng đợi sự kiện.
- **Nền tảng hiểu Freeze**: Nếu tác vụ hiện tại kéo dài (CPU-bound), cả hai hàng đợi chỉ biết chờ.

# `async/await` không phải đa luồng

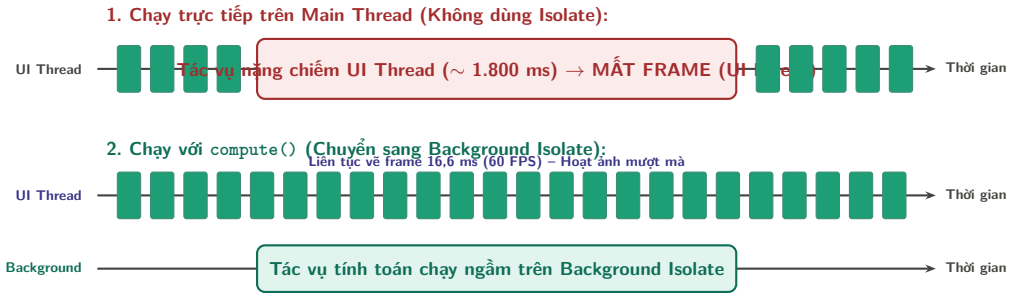
| Tiêu chí | **I/O-bound** (HTTP, file, DB) | **CPU-bound** (vòng lặp, parse, mã hoá) |
|---|---|---|
| **Thời gian đi đâu?** | Chờ thiết bị/mạng, CPU rảnh | Chính CPU của luồng Dart tính toán |
| `Future/await` **đủ chưa?** | **Đủ** – Event Loop rảnh khi chờ | **Không** – vẫn chiếm giữ UI thread |
| **Giải pháp** | Dùng `async/await` thông thường | **Phải dùng Isolate** / `compute` |

```dart
Future<int> heavy() async {
  return _heavyLoop(1500000000); // runs
      SYNCHRONOUSLY
} // on the UI thread!
```

**Chốt kiến thức quan trọng**

`async` **chỉ đổi cách CHỜ, không đổi NƠI CHẠY.** Code đồng bộ bên trong một hàm `async` vẫn chiếm dụng UI thread cho đến khi gặp từ khoá `await` đầu tiên.

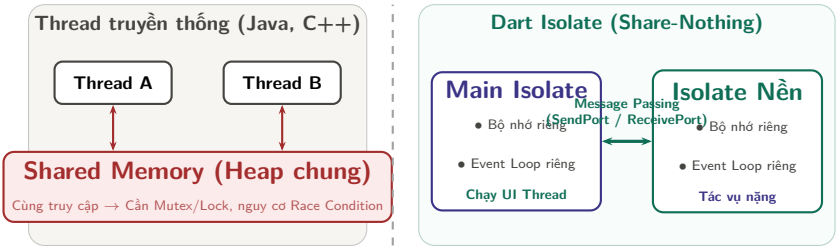
# Hệ quả trên dòng thời gian frame




Mỗi vạch xanh đại diện cho một frame hoàn thành trong 16,6 ms.

Nút 1 và Nút 2 ở phần Demo tái hiện đúng hai dòng thời gian này.

# Isolate: mô hình đồng thời của Dart




- **Không chia sẻ bộ nhớ (Share-nothing)**: Không sợ race condition hay dead-lock, không cần khoá.
- **Giao tiếp thuần qua thông điệp**: Chỉ truyền dữ liệu thông qua cơ chế sao chép hoặc chuyển giao.
- **Event Loop riêng**: Isolate nền bận tính toán nặng vẫn không làm đứng UI của Main Isolate.

# Giao tiếp giữa các isolate

Cơ chế mức thấp bằng cổng thông điệp mà `compute()` che giấu

```dart
Future<int> runWithSpawn(int n) async {
  final receive = ReceivePort(); // (1)
  await Isolate.spawn(_worker, [receive.sendPort, n]); // (2)
  final result = await receive.first as int;
  receive.close();
  return result;
}

void _worker(List<dynamic> args) {
  final port = args[0] as SendPort;
  port.send(_heavyLoop(args[1] as int)); // (3)
}
```

**(1)** Khởi tạo `ReceivePort` ở Main Isolate để nhận kết quả.

**(2)** Spawn isolate mới, gửi `SendPort` kèm tham số n.

**(3)** Worker tính toán và gửi kết quả về qua cổng.

**Cơ chế dữ liệu**

Dữ liệu vào được **sao chép** sang isolate nhận.
Kết quả chuyển về không sao chép nhờ `Isolate.exit`.
Không gửi được đối tượng gắn với UI.

# Ba cách dùng Isolate

| API | Mức độ | Dùng khi nào | Ghi chú đặc thù |
|---|---|---|---|
| `Isolate.spawn` | Thấp | Worker sống lâu, trao đổi nhiều lần | Tự quản lý `SendPort`/`ReceivePort` |
| `Isolate.run` | Trung bình | Tác vụ một lần, trả về `Future` (Dart $\geq$ 2.19) | Tự spawn, trả kết quả và tự huỷ |
| `compute()` | **Cao nhất** | **Tác vụ một lần trong Flutter** | Native: tương đương `Isolate.run`; **Web: chạy trên event loop hiện tại** |

```dart
final r1 = await Isolate.run(() => heavy(n)); // plain Dart
final r2 = await compute(heavy, n); // Flutter
```

**Lựa chọn của nhóm**

Nhóm chọn `compute()` vì phù hợp với tác vụ chạy một lần và API ngắn gọn, chuẩn Flutter.

# `compute()`: chữ ký và quy tắc

```
Future<R> compute<M, R>(
  ComputeCallback<M, R> callback, // runs in the background isolate
  M message, // single input value
);
```

1. **Quy tắc 1 (KHUYẾN NGHỊ top-level)**: `callback` nên là hàm top-level; không dùng closure bắt biến ngoài (tránh kéo theo tham chiếu widget/`BuildContext`).
2. **Quy tắc 2 (Đúng 1 tham số)**: Chỉ nhận đúng 1 đối số `message`. Nếu cần nhiều giá trị thì gói vào `List`, `Map` hoặc Data Class đơn giản.
3. **Quy tắc 3 (Dữ liệu gửi được)**: `message`, `callback` và kết quả phải **gửi được** giữa các isolate (kiểu cơ bản, danh sách, map, hoặc object thuần dữ liệu).
4. **Quy tắc 4 (Bắt lỗi an toàn)**: Lỗi ném ra trong isolate nền được **ném lại ở chỗ `await`** tại Main Isolate → bắt bằng `try/catch`.

# Luồng hoạt động của `compute()`




- **Cơ chế sao chép**: Tham số truyền vào được sao chép sang không gian bộ nhớ riêng của Isolate nền.
- **Trả kết quả & Tự huỷ**: Kết quả được chuyển giao về và isolate kết thúc.

# Chi phí và khi nào nên dùng

$T_{\text{compute}} \approx T_{\text{spawn}} + T_{\text{copy}} + T_{\text{task}} + T_{\text{return}}$ so với $T_{\text{main}} \approx T_{\text{task}}$ **(nhưng chặn UI)**

**NÊN DÙNG (Tác vụ CPU-bound nặng)**

- Phân tích chuỗi JSON lớn.
- Xử lý điểm ảnh, nén ảnh, resize ảnh.
- Mã hoá dữ liệu, hashing, nén file zip.
- Sắp xếp/lọc danh sách chục nghìn phần tử.
- Vòng lặp thuật toán tính toán nặng.
- *Tài liệu Flutter:* kéo dài **hơn vài mili-giây** (có nguy cơ rớt frame).

**KHÔNG NÊN DÙNG**

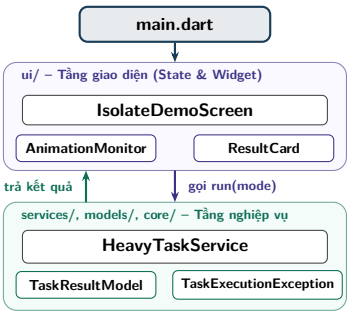
- Tác vụ quá nhẹ: chi phí spawn lớn hơn lợi ích mang lại.
- Tác vụ thuần I/O (gọi HTTP, đọc DB): dùng `async/await`.
- Tác vụ cần truy cập UI, đọc Widget tree hoặc `BuildContext`.

**CHỐT:** `compute` **giữ UI mượt, KHÔNG làm tác vụ chạy nhanh hơn.**

# Cạm bẫy và giới hạn

- **Closure bắt biến ngoài**: Truyền phương thức chứa tham chiếu `this` hoặc `BuildContext` → kéo theo trạng thái không gửi được qua isolate. Chỉ truyền dữ liệu tối thiểu.
- **Dữ liệu lớn gây tốn chi phí sao chép**: Đối tượng vào/ra quá đồ sộ làm tăng thời gian copy → chỉ gửi dữ liệu thô và nhận kết quả tinh gọn.
- **Plugin / Platform Channels**: Isolate nền không có sẵn cầu nối Native, cần gọi `BackgroundIsolateBinaryMessenger` nếu muốn gọi plugin nền tảng.
- **Nền tảng Flutter Web**: Web là môi trường đơn luồng JavaScript, `compute()` chỉ chạy trên event loop hiện tại, không tạo luồng mới.
- **Môi trường Debug**: Chạy ở chế độ Debug sẽ chậm hơn nhiều lần do có cờ kiểm tra → cần đo hiệu năng ở chế độ `--profile` hoặc `--release`.

# Kiến trúc ứng dụng và lý do tách tầng




- **Yêu cầu đề bài → Thiết kế**:
  - Hiệu ứng chuyển động liên tục → Tách thành widget `AnimationMonitor` chuyên dụng.
  - Hai nút nhưng **cùng một tác vụ** → Dùng chung đúng một hàm `_heavyLoop`.
  - Tách biệt trách nhiệm: UI không biết chi tiết isolate, toàn bộ nằm trong `HeavyTaskService`.
- Cấu trúc code sạch sẽ, tuân thủ nguyên tắc Single Responsibility.

# Code 1: hàm tác vụ nặng `_heavyLoop`

Tệp nguồn: `lib/services/heavy_task_service.dart`

```dart
int _heavyLoop(int iterations) { // (1)
  if (iterations <= 0) {
    throw ArgumentError('iterations > 0'); // (4)
  }
  var sum = 0;
  for (var i = 0; i < iterations; i++) { // (3)
    sum += (i % 7) * (i % 3);
  }
  return sum; // (2)
}
```

**Nhận xét:** Cùng 1 hàm cho cả 2 nút → so sánh công bằng.

**(1)** **Hàm top-level**, đúng 1 tham số – đúng khuyến nghị của `compute`.

**(2)** Vào/ra đều là `int` – **gửi được** giữa các isolate.

**(3)** Vòng lặp **CPU-bound**, không `await` – chạy ở Main Isolate sẽ **chiếm Event Loop**.

**(4)** `ArgumentError` ném trong isolate nền được **ném lại ở nơi** `await`.

# Code 2: `HeavyTaskService.run` – hai cách chạy

Tệp nguồn: `lib/services/heavy_task_service.dart`

```dart
Future<TaskResultModel> run(ExecutionMode mode,
    {int iterations = defaultIterations}) async {
  final stopwatch = Stopwatch()..start(); // (5)
  try {
    final int sum;
    if (mode == ExecutionMode.mainThread) {
      await Future<void>.delayed(
          const Duration(milliseconds: 50)); // (1)
      sum = _heavyLoop(iterations); // (2)
    } else {
      sum = await compute(_heavyLoop, iterations); // (3)
    }
    stopwatch.stop();
    return TaskResultModel(mode: mode, sum: sum,
        durationMs: stopwatch.elapsedMilliseconds);
  } catch (e) {
    throw TaskExecutionException('Task failed', e); // (4)
  }
}
```

**(1)** `delayed(50ms)` nhường Event Loop 1 nhịp để UI kịp vẽ trạng thái.

**(2)** Chạy trực tiếp trên Main Isolate – chiếm UI thread → **Freeze**.

**(3)** `compute` tạo isolate nền, chạy ngầm, Main Isolate vẫn vẽ frame.

**(4)** Bọc ngoại lệ thành `TaskExecutionException`.

**(5)** `Stopwatch` đo thời gian thực thi trọn vẹn cả chi phí spawn/copy.

# Code 3: giao diện – "cảm biến"và điều khiển

Tệp nguồn: `animation_monitor.dart` & `isolate_demo_screen.dart`

```dart
// AnimationMonitor (abridged)
const Row(children: [
  SizedBox(width: 48, height: 48,
    child: CircularProgressIndicator(strokeWidth: 5)), // (1)
  Expanded(child: LinearProgressIndicator(minHeight: 8)),
])
```

```dart
// _handleRun (abridged)
setState(() { _isRunning = true; }); // (2)
try {
  final result = await _service.run(mode); // (3)
  if (!mounted) return; // (4)
  setState(() => _result = result);
} on TaskExecutionException catch (e) { /* show error */ } // (5)
finally { if (mounted) setState(() => _isRunning = false); }
```

**(1)** Indicator vẽ lại **mỗi frame** – "cảm biến": Nếu UI thread bị chặn, vòng quay **lập tức đứng**.

**(2)** `setState` khoá cả 2 nút, ngăn bấm chồng.

**(3)** `await` ở UI chỉ chờ Future, không chặn Event Loop.

**(4)** Kiểm tra `mounted` an toàn.

**(5)** Bắt đúng `TaskExecutionException`; `finally` luôn mở khoá nút.

# Demo: cùng một tác vụ, hai cách chạy

**Kịch bản thực hiện (app thực tế):**

1. Quan sát `AnimationMonitor` xoay mượt mà.
2. Bấm **Nút 1 (Main Thread)**:
   - Vòng quay lập tức **đứng (Freeze)**.
   - UI hoàn toàn không phản hồi thao tác.
3. Bấm **Nút 2 (`compute`)**:
   - Vòng quay **vẫn xoay mượt mà**.
   - Kết quả hiện sau khi tính xong.






# Kết quả đo thực tế

| Chỉ số đo | Nút 1: Main Thread | Nút 2: `comput` |
|---|---|---|
| **Thời gian thực thi** | **1.842 ms** | **1.868 ms** |
| **Animation quay** | **Đứng hình** | **Mượt mà** |
| **UI Thread Frame** | **Block 1.842 ms** | **Ổn định $\leq$ 16,** |
| **Phản hồi thao tác** | **Đơ hoàn toàn** | **Phản hồi ng** |

[Chèn ảnh: Ảnh chụp kết quả đo thực tế
(DevTools Performance / CPU Profiler)
Hiển thị vạch đỏ nghẽn frame (Nút 1)
vs vạch xanh ổn định (Nút 2)]

- `compute` **không làm tính toán nhanh hơn**: Chênh lệch $\sim$ 26 ms do chi phí khởi tạo Isolate và sao chép dữ liệu.
- **Lợi ích cốt lõi là tính phản hồi (Responsiveness)**: Giải phóng hoàn toàn Main Thread để dựng hình liên tục.

# Kết luận

1. **Bản chất vấn đề**: Tác vụ CPU-bound nặng chạy đồng bộ trên Main Isolate làm **chiếm dụng Event Loop**, khiến sự kiện vẽ frame bị bỏ lỡ → gây ra UI Jank và UI Freeze.

2. **Bản chất giải pháp**: **Dart Isolate** giải quyết tận gốc bằng mô hình không chia sẻ đối tượng (bộ nhớ riêng, Event Loop riêng, message passing). Trong Flutter, hàm `compute()` là giải pháp gọn gàng và chuẩn mực nhất cho tác vụ chạy một lần.

3. **Đánh đổi kỹ thuật**: Tạo Isolate luôn tốn chi phí khởi tạo (spawn) và sao chép dữ liệu (copy) → **chỉ dùng khi tác vụ đủ nặng để có nguy cơ làm rớt frame**.

# Phân công và mã nguồn

| TT | Họ và tên | Nội dung phụ trách | Đóng góp |
|---|---|---|---|
| 1 | [Họ tên 1 - MSSV] | **Business Logic, Lý thuyết**: Xây dựng `task_execution_exception.dart`, `task_result_model.dart`, `heavy_task_service.dart`; Slide 3–14 | 50% |
| 2 | [Họ tên 2 - MSSV] | **UI, State, Demo**: Xây dựng `isolate_demo_screen.dart`, `animation_monitor.dart`, `result_card.dart`, `main.dart`, README; Slide 15–20 | 50% |

**Mã nguồn dự án trên GitHub:**

`https://github.com/[URL repo]`

# Cảm ơn thầy/cô đã lắng nghe!

## Xin mời thầy/cô và các bạn đặt câu hỏi

# Tài liệu tham khảo


1. Flutter Docs, *Concurrency and isolates*.
   https://docs.flutter.dev/perf/isolates
2. Flutter API, *compute function*.
   https://api.flutter.dev/flutter/foundation/compute.html
3. Dart Docs, *Concurrency in Dart*.
   https://dart.dev/language/concurrency
4. Flutter Docs, *Improving rendering performance*.
   https://docs.flutter.dev/perf/rendering-performance
5. Flutter Docs, *Flutter's build modes*.
   https://docs.flutter.dev/testing/build-modes